

GNC to the moon

...

Varun Ahlawat, Vy Dinh

ReIntroduction

Problem Statement:

How to get rockets to become more reusable by allowing to re-land with the usage of reinforcement learning models at the steering wheel?

Our Solution:

A Soft Actor Critic/Proximal Policy Optimization algorithm based model trained under a cloud computing environment.

Soft-Actor-Critic(SAC/PPO)

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

Algorithm 1 PPO, Actor-Critic Style

```
for iteration=1, 2, ... do
  for actor=1, 2, ...,  $N$  do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for
```

Solution Broken Down Further

- Have a physics (Julia) based simulation environment
- Have a main program with 1 P network & 4 Q networks.
- Have a worker process(es) run an agent that interacts that environment and collect SARs pairs.
- Use those SARs and update the primary P&Q and repass P&Q networks to workers.
- Rinse & Repeat

SAR- State, Action, Reward
P&Q - Policy & Q

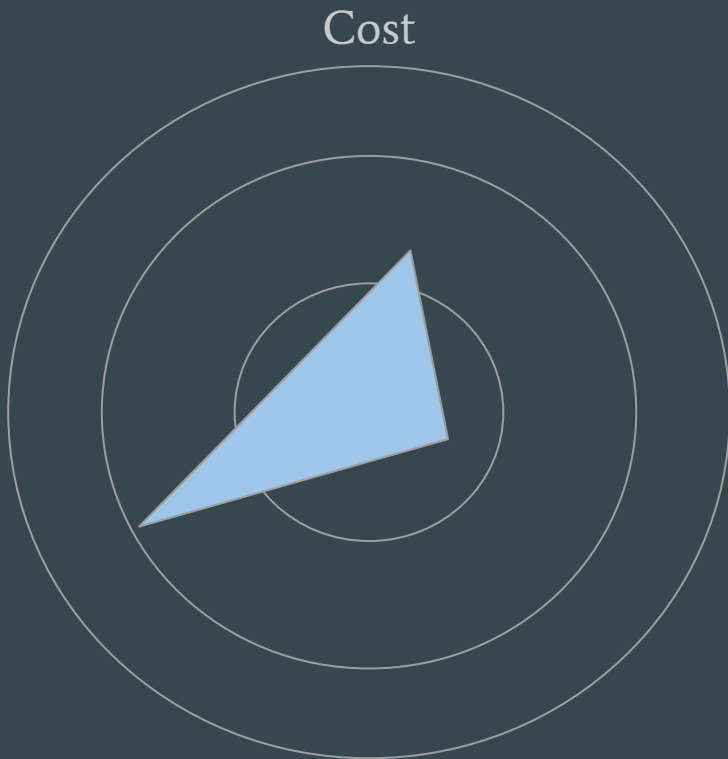
Current Scope & Leveraging Aspects of Cloud Computing



Current Scope & Leveraging Aspects of Cloud Computing

Focus: getting things working on atleast single decently powerful VM

Computing Power per instance



Cost

Size of Resource Pool

Julia Simulation

Julia Physics Simulation Architecture

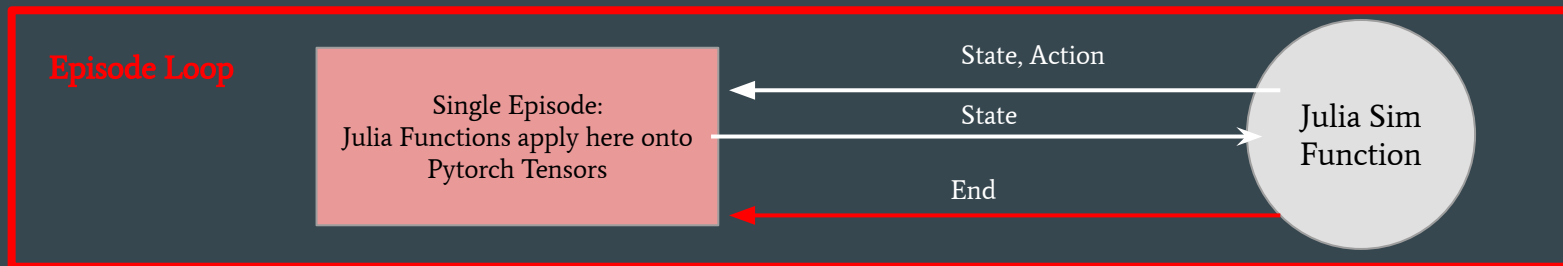
A Physics simulation engine written in Julia due to its C like performance.

Simulation is neither configured nor called from Julia.

Julia code is stateless.

The python training code contains the state configuration. In each communication state is being shared - REST like architecture.

The probability distribution of action for given state is then used in the loss functions of the policy(Actor) network, maximizing the Return(sum of rewards over an episode).



Python Driver Code

Design of Workers

Inputs:

- Episode_Count
- Target_Sim_Func
- Sync Events for Updating Weights, Execution Barrier, other multiprocessing elements etc.

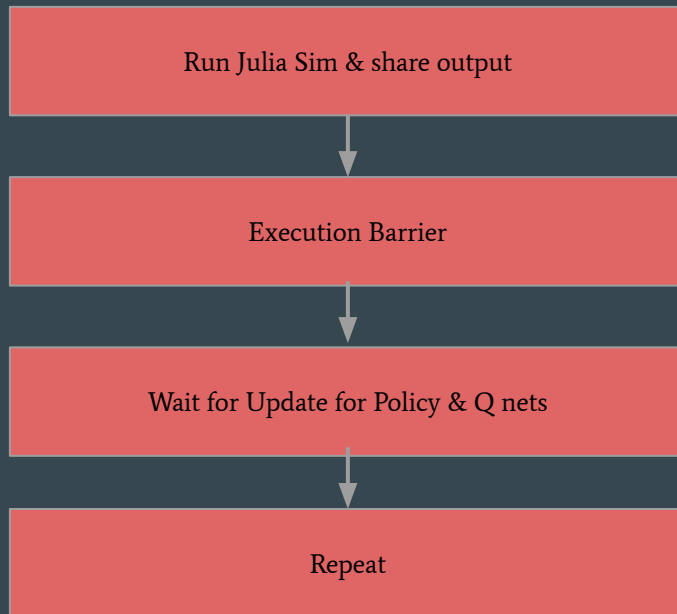
Outputs:

- {<sim_cnt: end_results>}

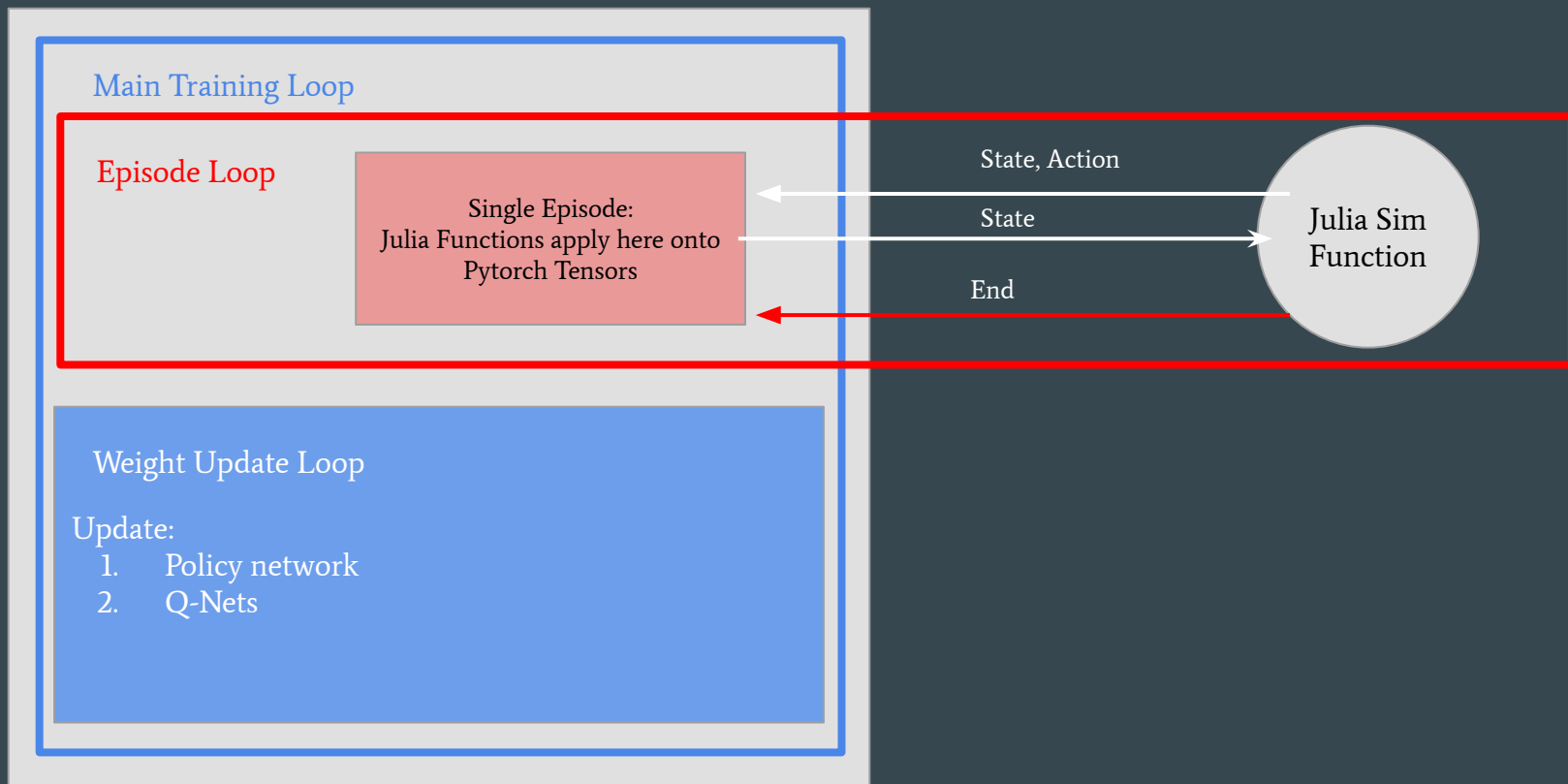
Methods:

- Run julia sim with shared Q-nets & Policy Nets

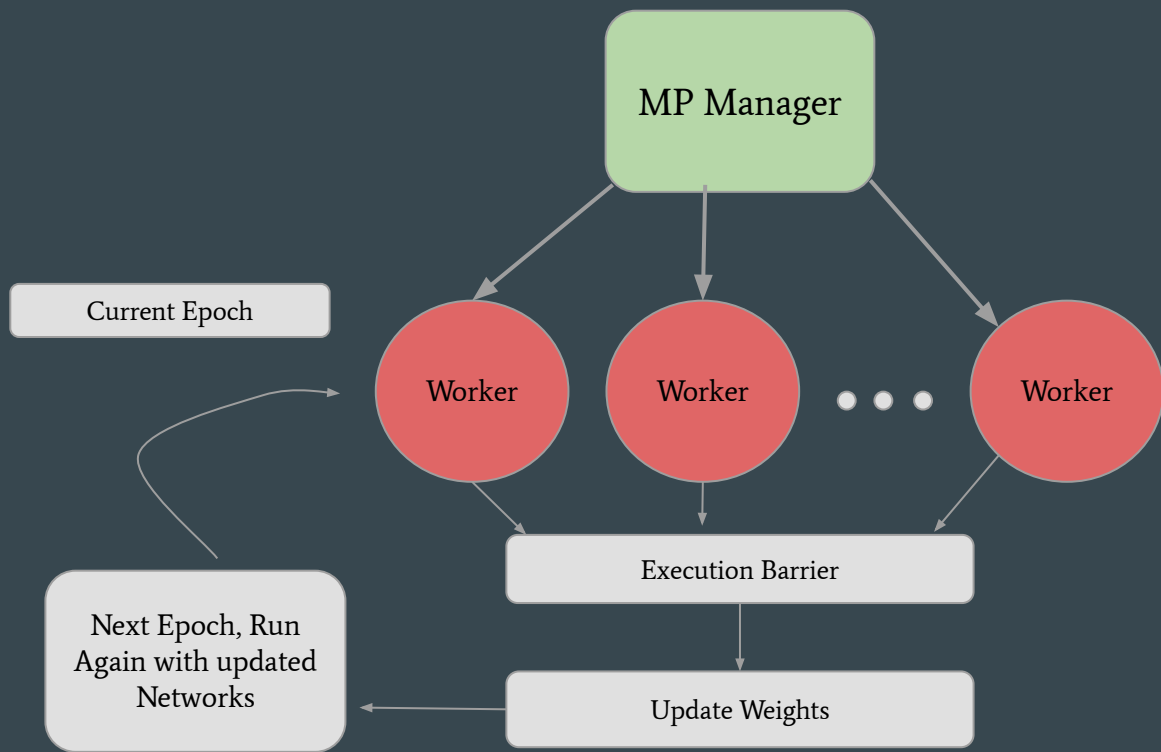
Julia Sim Execution



Execution of Driver Python Code



Execution of Driver Python Code



To avoid updating model parameters on incomplete distribution.

Single Worker's Performance

Testing Parameters:

local machine- Macbook ml

Main Program parameters

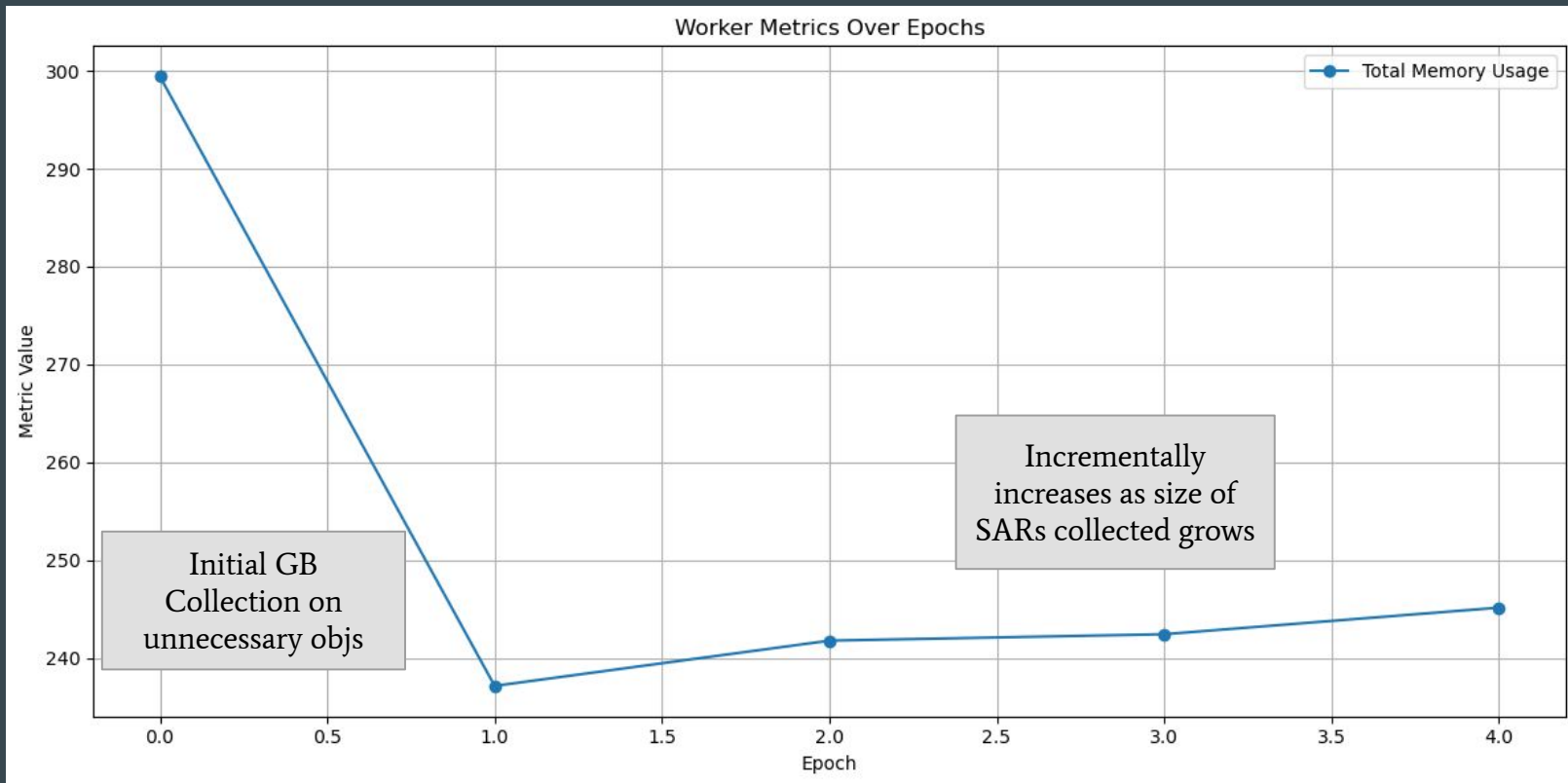
```
# Hyperparameters
EPISODE_CNT = 10 #Total Episodes per process
NUM_PROCESSES = 4 #Later dictate this to
EPOCH_CNT = 5
# EPLISON = 0.1 #For eplison decay of ea
EVALUATION_SELECTION = 10 #E
```

Note: NUM_PROCESSES = TOTAL NUM OF WORKERS;

This is a small experiment, don't expect it be accurate for long-term training

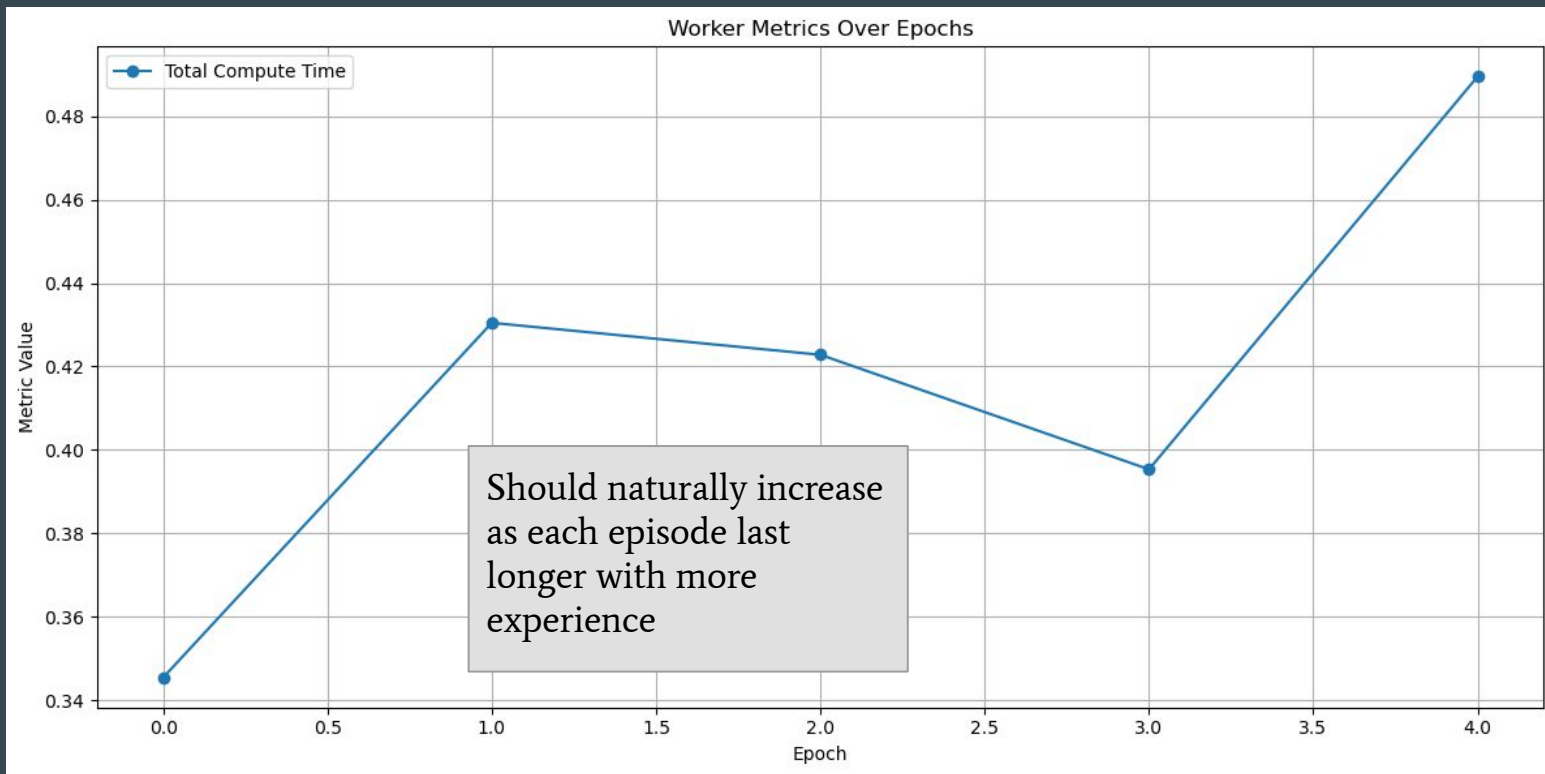
Single Worker Performance(Memory Consumption Over Epoches)

In
MB



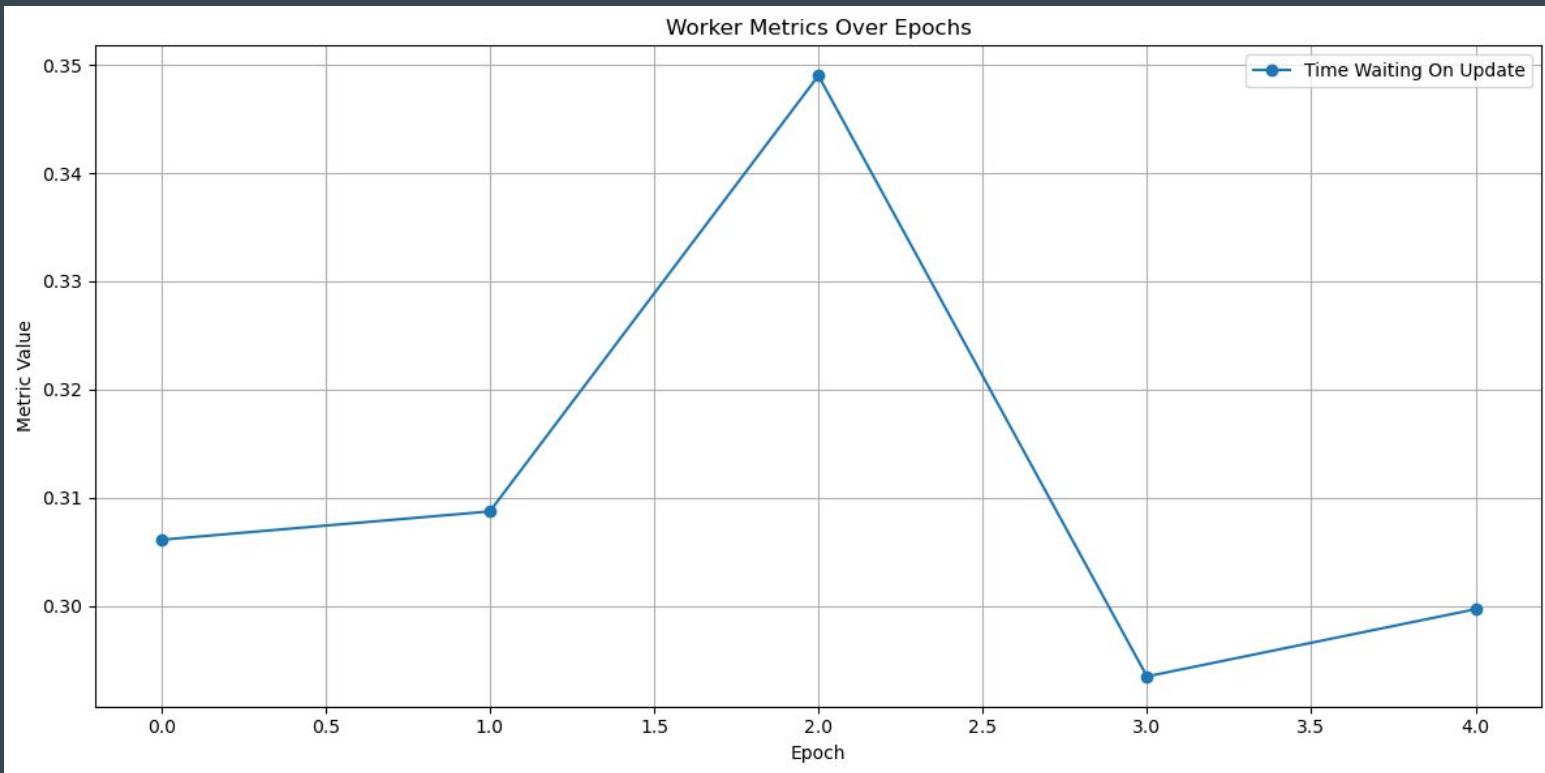
Single Worker Performance(Compute Time for each epoch)

In
Secs



Single Worker Performance(Wait Time for an Update)

In
Secs



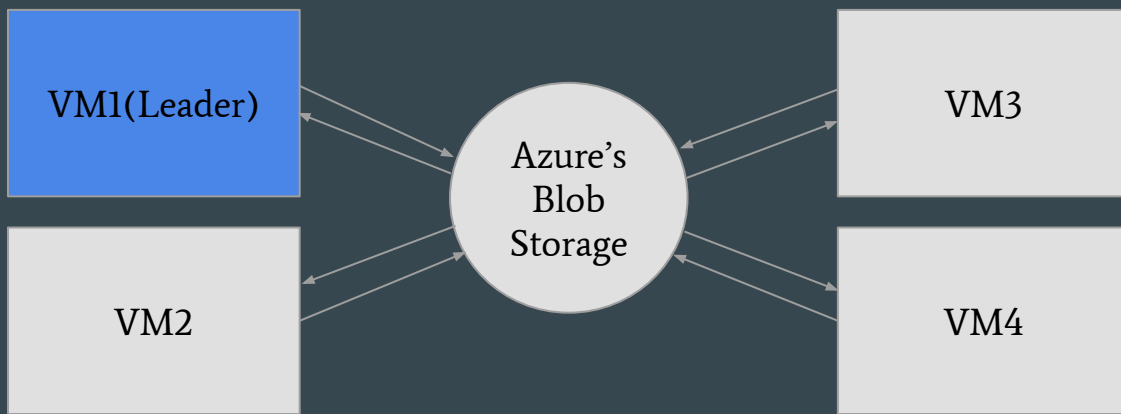
Aggregate Metrics Across all Workers Executing in Parallel

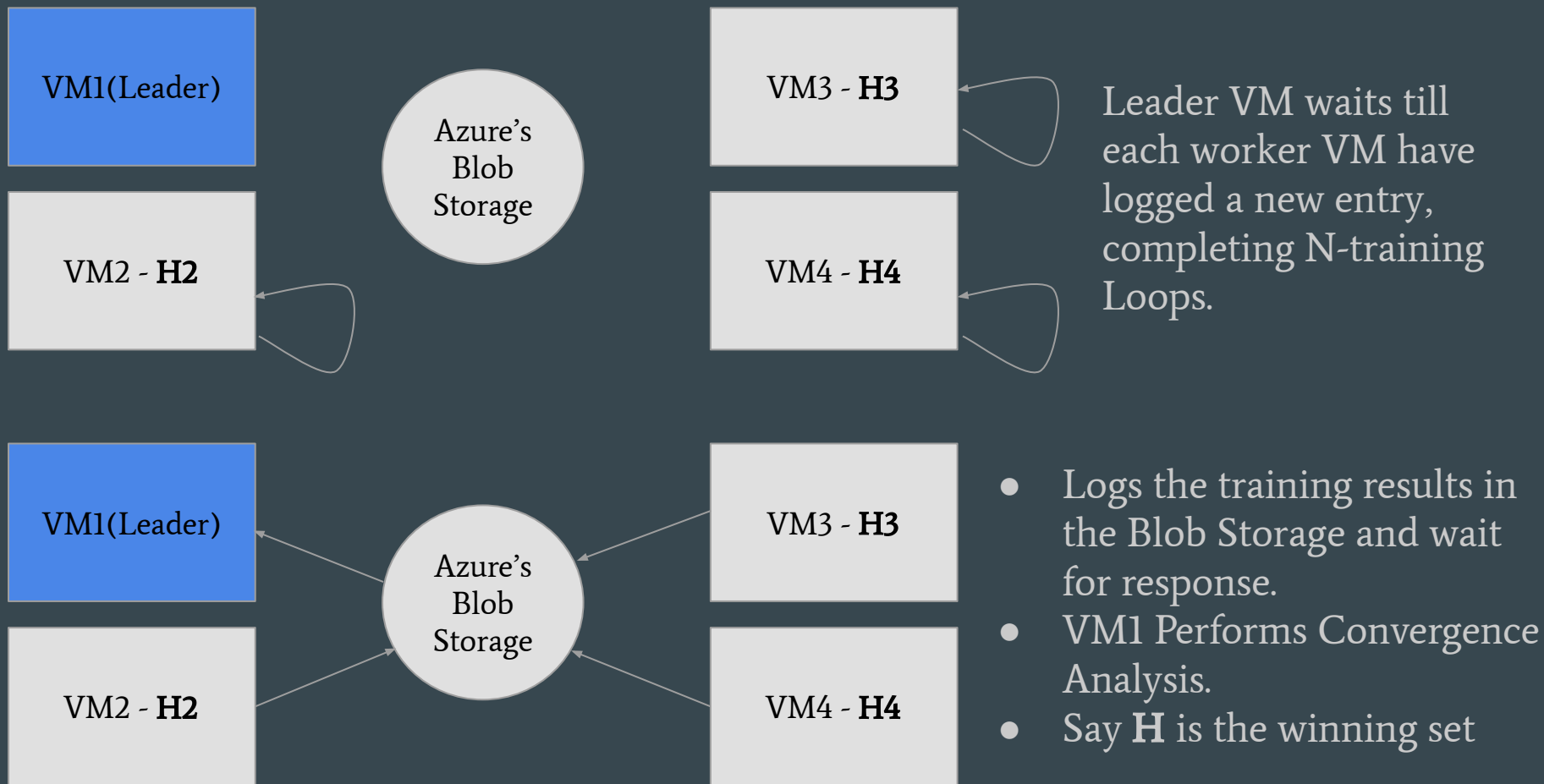
*for updates

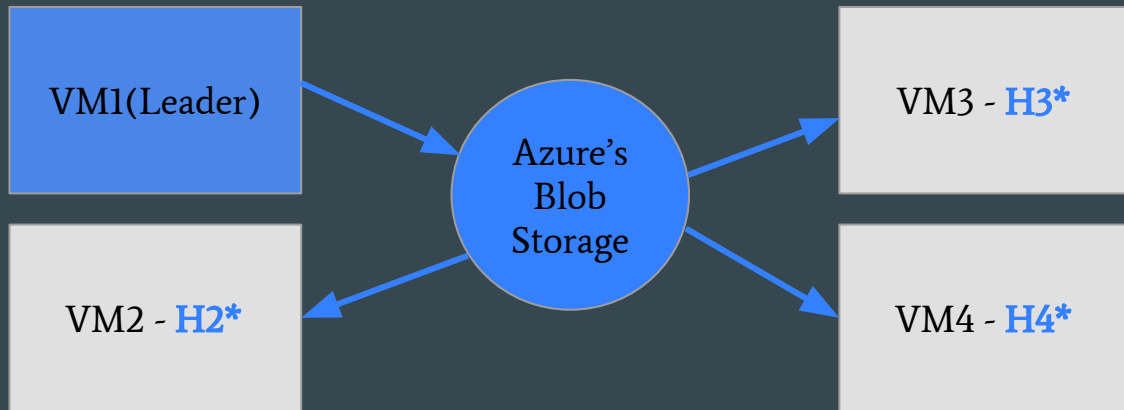
	total_execution_time	initial_mem_usage_MB	total_episodes	avg_compute_time_per_episode	avg_wait_time_per_episode
aggregate_function					
mean	21.431940	296.882812	50.0	0.042345	0.031140
std	6.435824	0.530714	NaN	0.000637	0.000004
min	13.937437	NaN	NaN	NaN	NaN
max	28.868538	NaN	NaN	NaN	NaN
sum	NaN	NaN	200.0	NaN	NaN

Azure Deployment Strategy - Overcoming Sparsity

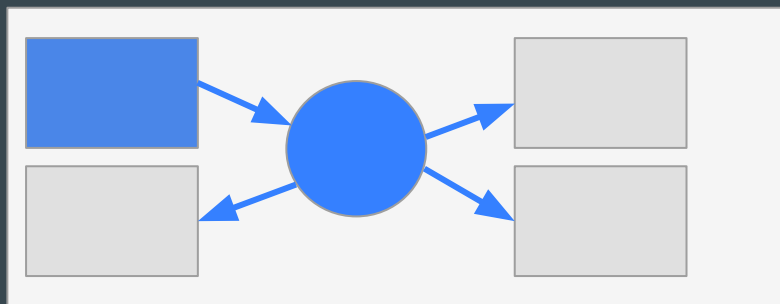
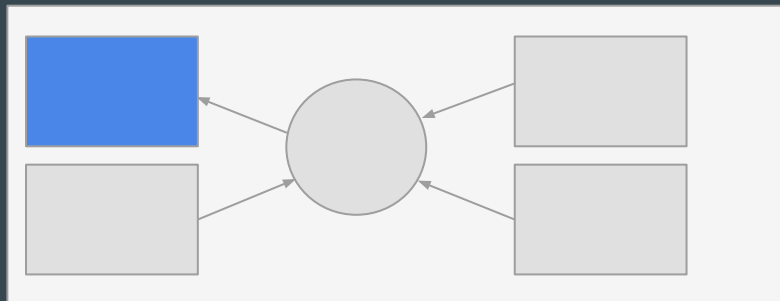
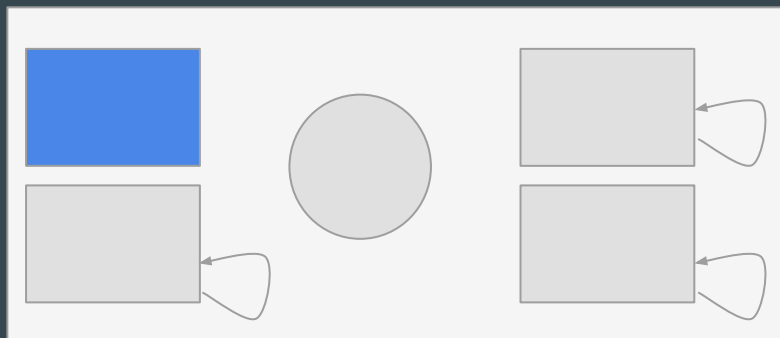
- Built a fully automated deployment system using ARM(Azure Resource Manager) templates to provision training infrastructure on demand.
- Centralized Blob storage across multiple VMs(this was our original plan):
 - Each VM trying a different set of VMs, logging in their results for N training loops.
 - After each N training loop, if one is converging better than others, let its hyperparameters be $\mathbf{H}$.
 - Neighborhood $\mathbf{H}$ be re-distributed across all the VMs resulting in significantly faster overall convergence.
 - When best set of $\mathbf{H}$ is found $\rightarrow$ invoke the parallel training: Use other VMs just for distributing the workers.







- VM1 adds generated noise and returns back perturbed H_s , one for each worker VM.



- VM1 adds progressively lesser noise in each cycle, eventually converging after a few cycles.
- This maximises the exploration highly efficiently.

Built In Entropy

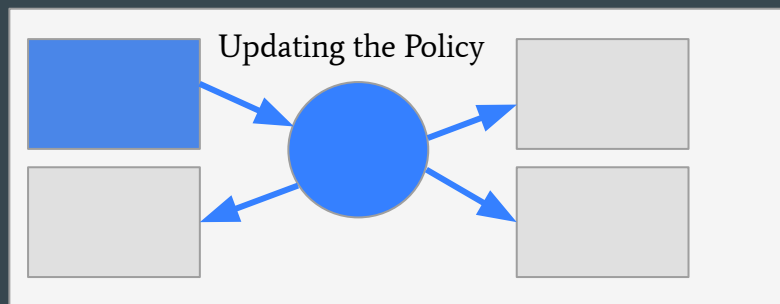
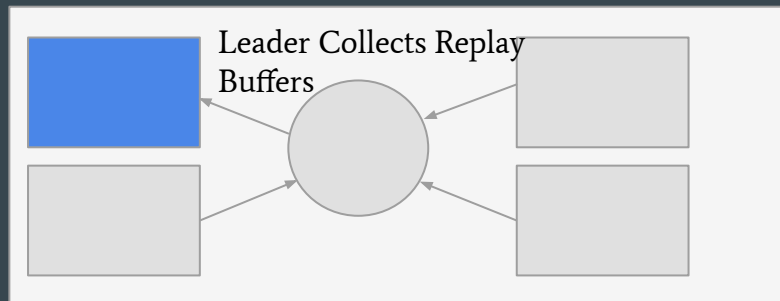
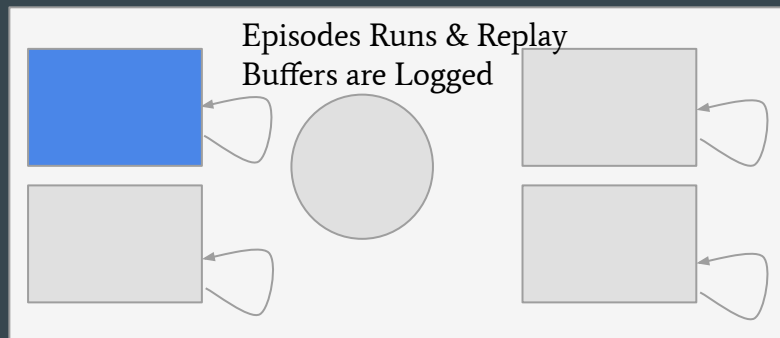
There's an entropy coefficient(ϵ) that randomizes the action taken by the network in the simulation.

It is higher in the initial episodes' evolution but reduces eventually.

Initializing the Episode with random behavior instead of deterministic policy over the decay of Entropy over time for maximising exploration.

All these methods are important because the rewards are insanely sparse due to continuous search space over continuous time.

- In the 22 D.O.F simulation environment written in Julia the convergence will be orders of magnitude slower than in OpenAI's Gymnasium's Lunar Lander.



After H convergence:

- All the worker VMs will stop running the training loops:
 - (Episodes + Training) → (Episode Workers)
- Leader will also run Episodes.
- Asynchronously all VMs will log a done message.
- Once Leader reads all done signals OR timeout hits: **Weights Updation Step is Invoked**

SETUP

```
1 {
2   "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
3   "contentVersion": "1.0.0.0",
4   "parameters": { ...
5 },
69 },
70 "variables": { ...
90 },
91 "resources": [
92   {
93     "type": "Microsoft.Network/networkSecurityGroups",
94     "apiVersion": "2019-02-01",
95     "name": "[variables('networkSecurityGroupName')]",
96     "location": "[variables('location')]",
97   > "properties": { ...
113 },
114 },
115 {
116   "type": "Microsoft.Network/virtualNetworks",
117   "apiVersion": "2019-02-01",
118   "name": "[variables('virtualNetworkName')]",
119   "location": "[variables('location')]",
120 > "properties": { ...
137 },
138 > "dependsOn": [ ...
140 ]
141 },
142 > { ...
154 },
155 > { ...
165 },
166 > { ...
176 },
177 > { ...
192 },
193 > { ...
222 },
223 > { ...
274 },
275 > { ...
299 }
300 ],
301 > "outputs": { ...
313 }
314 }
```

```
# Create VM configurations
for i in range(num_vms):
    config["vms"].append({
        "name": f"rl-training-vm-{i+1}",
        "size": vm_size,
        "training": {
            "total_timesteps": 500000,
            "learning_rate": 0.0003,
            "buffer_size": 1000000,
            "seed": 1000 + i, # Different seed for each VM
        }
    })

# Write the configuration to a file
with open(config_file, 'w') as f:
    json.dump(config, f, indent=2)

print(f"Created VM deployment configuration at {config_file}")
```

ARM Template contains all the variables and setup instructions. Shell Script sets env variables in each VM.

Everything is uploaded on the blob storage.

Once VMs are running, they'll automatically download our code, install dependencies, and training could be started.

Every follow up event will be asynchronous with timeouts for reliability.

Results & Findings

On Macbook Pro M2Pro, the
multiprocessing implementation had
total training time

58 minutes



Varun 07:40

```
rollout/
  ep_len_mean    225
  ep_rew_mean    268
time/
  episodes      1200
  fps           142
  time_elapsed  3489
  total_timesteps 498546
train/
```

time/fps

8000

```
● (venv) → rl_lunar_lander python visualize.py
2025-04-24 07:50:40.467 python[91710:4117958] +[IMKClient subclass
]: chose IMKClient_Modern
2025-04-24 07:50:40.467 python[91710:4117958] +[IMKInputSession su
bclass]: chose IMKInputSession_Modern
Episode finished with reward 236.28801755735364
Episode finished with reward 296.1285448969254
Episode finished with reward 306.7167354581041
Episode finished with reward 296.5235542711806
```

Run ↑



SAC_lunar_lander_1745475265/SAC_1

Smoothed

142

Value

142

Step

498,546

Relative

58.15 min

498546 X

VM Results Comparison(On C.P.U)

Resource Utilization of a Worker:

> deployment work is in progress

Resource Utilization of the VM:

> deployment work is in progress

What we had to skip due to Time & Money constraints:

- Julia Simulation → Almost Finished Implementation:
 - Physics engine has been written.
 - Configuration design and battle testing remain.
 - We chose to skip this part also because we didn't have any idea how long it'll take to train due to significantly sparser reward space and having no benchmarks to compare with.
- Distributed CFD Simulations
- Two SAC's setup, one additional Wind Adversary to train against the main P network, knocking the rocket off using limited wind energy.
- Multi VM Setup:
 - Distributed
- Frontend for monitoring workers and seeing live metrics.
- Distributed GPU training - Money for more expensive VM setups.

Guidance Navigation Control using Reinforcement Learning Control Systems Paradigm

Controlling a rocket as precisely as SpaceX, enabling propulsive landing. Fully generalizable solution for any rocket.

INTRODUCTION

ABSTRACT

Control is at the forefront of technology, many systems use deterministic models for this such as MPC, LQR, PID for control. Our goal is to teach a Neural Network, trained through reinforcement learning, to be able to autonomously control a 3D rocket through 2DOF thrust vectoring, a control method whereby some actuator's thrust vector is rotated by some angle thus changing attitude. To accomplish this, a few things are needed: a computer to be able to intake necessary sensor data and output actuator signals. A proper state estimation algorithm to mitigate sensor noise and a near 1 to 1 physics environment for the neural network to train.

ALGORITHMS

- Reinforcement Learning: Soft Actor Critic
- State Estimation: Particle Filter
- Force Coefficient CFD: K Omega SST

FORMULAE

- Critic Loss:**
$$\mathcal{L}_Q(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\left(Q_\theta(s_t, a_t) - y \right)^2 \right]$$
- Target Value:**
$$y = r_t + \gamma \mathbb{E}_{a_{t+1} \sim \pi(\cdot | s_{t+1})} \left[\min_{i=1,2} Q_{\hat{\theta}_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1} | s_{t+1}) \right]$$
- Target Critic Update:**
$$\hat{\theta}_i \leftarrow \tau \hat{\theta}_i + (1 - \tau) \hat{\theta}_i$$
- Policy Loss:**
$$\mathcal{L}_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi} \left[\alpha \log \pi_\phi(a_t | s_t) - Q_\theta(s_t, a_t) \right]$$
- Reparameterization Trick:**
$$a_t = \mu_\phi(s_t) + \sigma_\phi(s_t) \cdot \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I)$$

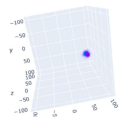
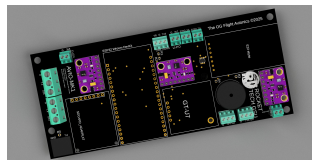
$$\pi_\phi(a_t | s_t) = \mathcal{N}(a_t | \mu_\phi(s_t), \sigma_\phi^2(s_t))$$
- Turbulent KE Eq:**
$$\frac{\partial(\rho \omega)}{\partial t} + \frac{\partial(\rho \omega u)}{\partial x_j} = \alpha \frac{\omega}{k} P_k - \beta \rho \omega^2 + \frac{\partial}{\partial x_j} \left[\left(\mu + \sigma_\omega \mu_t \right) \frac{\partial \omega}{\partial x_j} \right] + 2(1 - F) \rho \sigma_{\omega 2} \frac{1}{\omega} \frac{\partial k}{\partial x_j} \frac{\partial \omega}{\partial x_j}$$
- Dissipation rate w formula:**
$$\frac{\partial(\rho k)}{\partial t} + \frac{\partial(\rho k u)}{\partial x_j} = P_k - \beta^* \rho k \omega + \frac{\partial}{\partial x_j} \left[\left(\mu + \sigma_k \mu_t \right) \frac{\partial k}{\partial x_j} \right]$$

RESEARCH

ENGINEERING

Our avionics team built **AvioMk 1**, The first generation PCB board for active avionics, to capture and feed in 34 sensor data streams to the Policy Network. It also controls the ESC for BLDC motor, supports RF communication, GPS, camera trigger, pyro charges' ejection, and up to two high-powered servo motor control.

CAD, simulations, and fabrication of the static setups, avionics bay, high powered test rockets, ...

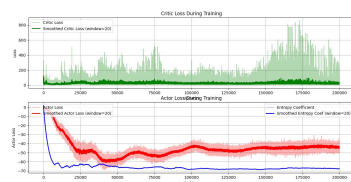


SOFTWARE ENGINEERING

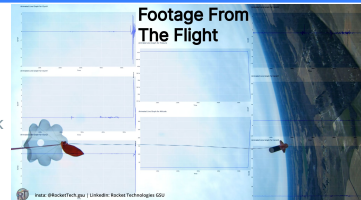
- Embedded C++** Flight Software and Sensor Libraries.
- Feed forward implementation of the policy network on the MCU(embedded system) on AvioMk 1
- Event Detection and Particle Filter algorithm for state estimation.
- Physics Simulation implemented from scratch in **Julia** for C like performance.
- Interfaced the simulation with Stablebaseline's **SAC Implementation** in Python.
- Parallelized Cloud **Training** performed on a cluster of 4 NVIDIA's A100 GPUs.

RESULTS

- Flight software, event detection tested on actual rocket's flight.
- The Neural Network, trained in OpenAI's Gymnasium, reached global optima in 150,000[~] steps.



CONCLUSIONS



CONCLUSION

Reinforcement Learning is here to replace all the conventional control systems(both optimal and heuristical). Ability of "RL Agent" to adapt to severe unlikely conditions is much higher than conventional control systems. Specialized simulation environment is required for each RL based control agent's training, which is the only remaining bottleneck.

ACKNOWLEDGEMENTS

- Student Org: **Rocket Technologies GSU**
- Stable Baseline's Soft Actor Critic Learning Implementation
- Brian Nee, Bnee2@student.gsu.edu, Static Attitude Control Test Setup & Design
- Pratham Thakkar, Thrust Vector Control
- A.Y. Sodipe, Landing Legs Setup

CONTACT INFO

<https://rocket-tech-gsu.github.io>
yahlawati1@student.gsu.edu, +1 470 599 5159
vwulfekuhle1@student.gsu.edu
cmawell21@student.gsu.edu